

Prueba - Modelo predictivo en Apache Spark MLlib

Clasificación binaria de transacciones riesgosas

1. Objetivo del desafío

El objetivo es construir un modelo supervisado con Apache Spark MLlib que estime la probabilidad de que una transacción de venta sea riesgosa. El flujo debe preparar features y label, dividir los datos, entrenar un clasificador, generar predicciones y evaluar su desempeño con métricas de clasificación.

2. Preparación y validación del dataset

El archivo original contiene 200 registros. Luego de validar campos obligatorios, tipos numéricos y fecha/hora, quedan 132 registros utilizables.

2.1 Definición razonada de la columna label

Se define una transacción como riesgosa (`label = 1`) cuando cumple al menos una de estas condiciones: (a) `Monto_Total > 7.000`; o (b) ocurre en madrugada (`Hora < 6`) y `Monto_Total > 4.000`. La regla combina una señal de magnitud monetaria con una señal temporal inusual, coherente con el caso de transacciones atípicas y horarios anormales. Las demás transacciones reciben `label = 0`.

Clase	Cantidad	Interpretación
0 - normal	96	Transacción que no cumple la regla de riesgo.
1 - riesgosa	36	Monto alto o combinación de madrugada y monto relevante.

2.2 Construcción de features

Las variables categóricas Sucursal y Producto se procesan con `StringIndexer` y `OneHotEncoder`. Las variables numéricas Cantidad, Precio_Unitario, Monto_Total y Hora se incorporan al vector de entrada con `VectorAssembler`. Finalmente, `StandardScaler` estandariza la representación antes del entrenamiento.

Etapas	Herramienta MLlib	Propósito
Indexación	StringIndexer	Convierte categorías a índices reproducibles.
Codificación	OneHotEncoder	Evita tratar categorías como magnitudes ordinales.
Ensamblaje	VectorAssembler	Genera la columna <code>features</code> .
Escala	StandardScaler	Homogeneiza escalas para el modelo.
Orquestación	Pipeline	Encadena preprocesamiento y entrenamiento.

La solución incorpora las validaciones solicitadas en la rúbrica mediante `printSchema()`, `show()` y conversión explícita de tipos. También utiliza `handleInvalid="keep"` en los indexadores para hacer el pipeline más robusto frente a categorías no vistas.

3. Modelo supervisado en Spark MLlib

Se selecciona `LogisticRegression` para construir una línea base probabilística de clasificación binaria. El dataset se divide con `randomSplit([0.75, 0.25], seed=42)`. El Pipeline se ajusta únicamente con el conjunto de entrenamiento y luego se aplica al conjunto de prueba.

Flujo implementado:

```
Dataset -> StringIndexer -> OneHotEncoder -> VectorAssembler -> StandardScaler
        -> LogisticRegression -> prediction / probability
```

La evidencia solicitada se obtiene con ``predicciones.select('label','prediction','probability').show()``. Además, la solución muestra Sucursal, Producto, Monto_Total y Hora para facilitar la interpretación operativa.

4. Evaluación del modelo

Se calculan tres métricas: accuracy, F1-score y areaUnderROC. Accuracy resume la proporción total de aciertos; F1 equilibra precision y recall; AUC evalúa la capacidad de ranking del modelo a través de distintos umbrales.

El entorno utilizado para preparar este documento no dispone de PySpark, por lo que las métricas definitivas deben obtenerse ejecutando el notebook en Google Colab, Databricks o un entorno Spark. Para verificar la lógica del dataset y del modelo se realizó una validación local equivalente, cuyos resultados de referencia fueron:

Métrica	Resultado de referencia
Accuracy	0.909
F1-score	0.857
AreaUnderROC	0.981
Matriz de confusión	[[21, 3], [0, 9]]

Estos valores son una comprobación de consistencia y no sustituyen la evidencia de ejecución PySpark. La entrega incluye un notebook Colab que calcula automáticamente las métricas Spark definitivas.

5. Justificación técnica y estratégica (máx. 200 palabras)

Se eligió LogisticRegression porque el problema es de clasificación binaria y el modelo entrega una probabilidad interpretable de riesgo, útil para priorizar transacciones según umbral. También constituye una línea base eficiente y reproducible dentro de Spark MLlib. Accuracy permite medir el rendimiento global, pero no es suficiente por sí sola cuando la clase riesgosa es menos frecuente. Por ello se incorpora F1-score, que equilibra precision y recall, y areaUnderROC, que mide la capacidad de separar riesgosas de normales para distintos umbrales. En negocio, un falso negativo puede ser más costoso que un falso positivo, por lo que la matriz de confusión y el recall de la clase 1 deberían acompañar la decisión operativa. En una siguiente versión se recomienda usar CrossValidator y ParamGridBuilder, comparar con RandomForestClassifier, incorporar clima, promociones, inventario e historial temporal, y ajustar el umbral de clasificación según el costo de error. El uso de Pipeline permite conservar exactamente las mismas transformaciones durante entrenamiento y scoring, reduciendo inconsistencias y facilitando la puesta en producción.

6. Mejoras futuras propuestas

- Validación cruzada con CrossValidator y búsqueda de hiperparámetros.
- Comparación con RandomForestClassifier y DecisionTreeClassifier.
- Incorporación de variables de clima, promociones, inventario y calendario.
- Features temporales: día de semana, franja, ventas previas y medias móviles.
- Ajuste del umbral para priorizar recall de transacciones riesgosas.
- Persistencia del Pipeline entrenado y monitoreo de drift.

7. Conclusión

La solución cumple el flujo completo solicitado: construcción razonada de la columna label, preparación robusta de features mediante herramientas de Spark MLlib, división entrenamiento/prueba, entrenamiento con Pipeline, predicción probabilística y evaluación con múltiples métricas. La propuesta también conecta el desempeño del modelo con el objetivo operacional de detectar transacciones potencialmente anómalas.

8. Archivos entregados

- Prueba_Modelo_Predictivo_Spark_MLlib.py - script PySpark comentado.
- Prueba_Modelo_Predictivo_Spark_MLlib_Colab.ipynb - notebook listo para Colab.
- ventas_simuladas_mllib.csv - dataset limpio con Hora y label.
- metricas_referencia_local.json - resultados de validación local de consistencia.